Overview

Now that you've analyzed the Wild Rydes codebase, it's time to run the actual .NET modernization. In this section, you'll use ATX Custom to convert the .NET Framework 4.8.1 application to cross-platform .NET 8.

The ATX agent will analyze the project structure, create a transformation plan, convert the code, and validate the results — all in a single execution.

EXERCISE 1: VERIFY ATX IS READY

1. Navigate to the wildrydes directory

```
cd /Workshop/wildrydes
```

2. Verify ATX CLI is available

```
atx custom def list
```

You should see the list of available AWS-managed transformations as before.

> **Troubleshooting**
> If you get an authentication error or wrong region, your session may have expired or the region may not be set in this terminal. Refer back to Setting Up AWS Transform Custom CLI Exercise 2 to reconfigure your credentials and region.

EXERCISE 2: CREATE A TRANSFORMATION DEFINITION

Since there is no AWS-managed transformation specifically for .NET Framework to .NET 8 conversion, the ATX agent will guide you through creating a **custom transformation definition** first.

1. Start the ATX interactive session

```
atx -t
```

> **Flags**
> - `-t` — Trust all tools (no tool prompts)

2. Tell the agent to create a transformation definition with the following detailed prompt:

```
Create a transformation definition to convert a .NET Framework 4.8.1 (C#) application to .NET 8 (C# 12) for cross-platform Linux deployment.

Source stack:
- ASP.NET MVC 5.3 with Razor Views
- Entity Framework 6.5.1 with SQL Server LocalDB (System.Data.SqlClient)
- NuGet packages.config format
- AWSSDK v4 (CognitoIdentity, LocationService, SecurityToken)
- Newtonsoft.Json and System.Text.Json
- Bootstrap 5.3, jQuery 3.7, Modernizr
- Windows-only hosting (IIS, Global.asax, Web.config)

Target stack:
- ASP.NET Core MVC with Razor Views (or Blazor Server)
- Entity Framework Core 8 with Microsoft.Data.SqlClient
- SDK-style .csproj with PackageReference format
- AWSSDK v4 updated packages compatible with .NET 8
- System.Text.Json (drop Newtonsoft.Json if possible)
- Bootstrap 5.3 (keep), drop Modernizr
- Cross-platform hosting (Program.cs, appsettings.json)

Key conversion areas:
1. Project file: Legacy .csproj to SDK-style targeting net8.0
2. Web framework: Global.asax + Web.config to Program.cs + appsettings.json
3. MVC: ASP.NET MVC 5 to ASP.NET Core MVC
4. ORM: Entity Framework 6 to Entity Framework Core 8 (DbContext, migrations, connection strings)
5. Database: System.Data.SqlClient to Microsoft.Data.SqlClient
6. AWS SDKs: Verify and update for .NET 8 compatibility
7. Package management: packages.config to PackageReference
8. Static assets: BundleConfig to modern bundling or keep as-is

The source code is located at ./sourceCode/. Read the source files to understand the current structure before creating the definition.
The ATXDocumentation/ folder contains the codebase analysis output from the previous step. Use it as reference for architecture, dependencies, and technical debt.
```

3. Once the definition is created, you should see the validation criteria and options:

```
The validation criteria include 20 exit conditions covering build success, route
preservation, database schema compatibility, AWS integration, security fixes, and
cross-platform Linux deployment. You can review and edit the transformation definition
directly at:

~/.aws/atx/custom/<session_id>/artifacts/tp-staging/transformation_definition.md

What would you like to do next? You can:
1. Apply this transformation to your code repository
2. Modify the transformation definition
3. Publish it to the registry for reuse
4. Review the full transformation definition

>
```

4. Publish the transformation definition to the registry for reuse

```
Publish it
```

5. The agent will ask for a name and description. Provide the following:

```
name: DotNet-Framework-4.8-to-DotNet-Core-8-Migration, description: good as your suggested
```

6. Verify the transformation definition was created

```
List and view detail of the transformation "DotNet-Framework-4.8-to-DotNet-Core-8-Migration" in the registry
```

You should now see your custom transformation definition listed.

Expected result (trimmed):

```
Here's the full transformation definition for
"DotNet-Framework-4.8-to-DotNet-Core-8-Migration":

Objective: Migrate a .NET Framework 4.8.1 ASP.NET MVC 5 web application (C#) to
.NET 8 (C# 12) with ASP.NET Core MVC ...
Entry Criteria: 10 conditions | Implementation: 10 phases, 48 steps
Validation / Exit Criteria: 20 conditions
```

7. Type `/quit` to exit the ATX session

```
/quit
```

EXERCISE 3: RUN THE CODE CONVERSION AND REVIEW RESULTS

Now that the transformation definition is published, you can execute it against the Wild Rydes source code using the `-n` flag to reference it by name.

1. Execute the transformation

```
atx custom def exec -n DotNet-Framework-4.8-to-DotNet-Core-8-Migration -p ./sourceCode -c "dotnet build" -t
```

> **Flags**
> - `-n` — Name of the transformation definition in the registry
> - `-p ./sourceCode` — Path to the .NET source code directory
> - `-c "dotnet build"` — Build command to validate the conversion
> - `-t` — Trust all tools (no tool prompts)

Expected result:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                           Region: us-east-1                              │
└──────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────┐
│  🟡 All tools are now trusted. ATX will execute tools without asking for  │
│                             confirmation.                                │
└──────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────┐
│ 🟡 You are using a user-created transformation which has not been reviewed │
│                                by AWS.                                   │
└──────────────────────────────────────────────────────────────────────────┘

📝 Conversation log: /home/participant/.aws/atx/custom/<session_id>/logs/<timestamp>-conversation.log
📊 Monitor progress: tail -f /home/participant/.aws/atx/custom/<session_id>/logs/<timestamp>-conversation.log

If interrupted, you can resume this conversation by running: atx --conversation-id <session_id>

I'll begin orchestrating the .NET Framework 4.8.1 to .NET 8 ASP.NET Core migration.

## Phase 1: Understanding Transformation
### Step 1: Ask User For Additional Feedback

Do you have any additional preferences I should take into consideration beyond what is
described in the Transformation Definition?

>
```

> **Timing**
> The code conversion time varies based on the source code complexity and can take **1 to 4 hours**.

> **Resuming if interrupted**
> If the session gets interrupted (e.g., network issue, terminal closed), you can resume it:
>
> ```
> atx --resume
> ```
>
> Or resume a specific conversation using the conversation ID shown at the start of the session:
>
> ```
> atx --conversation-id <conversation_id> -t
> ```
>
> Conversations can be resumed within 30 days of creation.

2. The ATX agent may ask you several questions during the conversion. Simply respond with `go` each time.

```
go
```

> **Tip**
> The agent may prompt you multiple times. Just keep responding with `go` until the conversion completes.

3. Once the conversion is done, you should see the validation result

```
Validation successful - All 20/20 exit criteria passed. Complete validation summary
has been written to ~/.aws/atx/custom/<session_id>/artifacts/validation_summary.md.

Do you have any feedback or questions about the transformation execution?

>
```

4. Type `/quit` to exit the ATX session

```
/quit
```

5. Check the git log for changes made by the transformation

```
git log --oneline
```

You should see one or more commits made by the ATX agent with the conversion changes.

6. Review the converted project file

```
cat sourceCode/wildrydes.net/wildrydes.net.csproj
```

The project file should now target .NET 8 instead of .NET Framework 4.8.1:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <RootNamespace>wildrydes.net</RootNamespace>
    ...
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.11" />
    <PackageReference Include="Microsoft.Data.SqlClient" Version="5.2.2" />
    <PackageReference Include="AWSSDK.CognitoIdentity" Version="4.0.0.15" />
    <PackageReference Include="AWSSDK.LocationService" Version="4.0.1" />
    <PackageReference Include="AWSSDK.SecurityToken" Version="4.0.1.6" />
  </ItemGroup>
</Project>
```

7. Check for any remaining issues

```
git status
```

The converted project structure now looks like this:

```
sourceCode/
├── wildrydes.net.sln                # Solution file (updated)
└── wildrydes.net/
    ├── Program.cs                   # NEW — ASP.NET Core entry point (replaces Global.asax)
    ├── appsettings.json             # NEW — Configuration (replaces Web.config)
    ├── wildrydes.net.csproj         # SDK-style, targeting net8.0 with PackageReference
    ├── Dockerfile                   # Updated — Linux-based multi-stage .NET 8 build
    ├── .dockerignore
    ├── Controllers/                 # Refactored — constructor-based DI, ASP.NET Core APIs
    │   ├── HomeController.cs
    │   ├── RideController.cs
    │   ├── UnicornController.cs
    │   └── UserController.cs
    ├── Models/                      # Updated — EF Core compatible
    │   ├── RideModel.cs
    │   ├── UnicornModel.cs
    │   └── UserModel.cs
    ├── Context/                     # Updated — EF Core 8 DbContext
    │   └── DefaultContext.cs
    ├── Decorators/                  # Updated — ASP.NET Core filter pattern
    │   └── Protected.cs
    ├── Views/                       # Updated — Tag Helpers, _ViewImports
    │   ├── _ViewImports.cshtml      # NEW — Tag Helper imports
    │   ├── _ViewStart.cshtml
    │   ├── Home/
    │   ├── Ride/
    │   ├── Unicorn/
    │   ├── User/
    │   └── Shared/
    └── wwwroot/                     # NEW — Static files (replaces Content/ and Scripts/)
        ├── Content/                 #   CSS, images
        └── Scripts/                 #   JavaScript
```

Key changes summary:

Before (.NET Framework 4.8.1)	After (.NET 8)
`Global.asax` + `App_Start/`	`Program.cs`
`Web.config`	`appsettings.json`
Legacy `.csproj` + `packages.config`	SDK-style `.csproj` + PackageReference
Entity Framework 6 + `System.Data.SqlClient`	EF Core 8 + `Microsoft.Data.SqlClient`
HTML Helpers in Razor Views	Tag Helpers + `_ViewImports.cshtml`
`Content/` + `Scripts/` at project root	`wwwroot/Content/` + `wwwroot/Scripts/`
Windows Server Core IIS Dockerfile	Linux-based multi-stage .NET 8 Dockerfile
`Modernizr`, `WebGrease`, `Newtonsoft.Json`	Removed (using built-in `System.Text.Json`)

EXERCISE 4: VERIFY SECURITY AND DEBT IMPROVEMENTS

The original codebase analysis (from `ATXDocumentation/technical-debt-report.md`) identified **6 high-severity** and **5 medium-severity** issues. Let's verify which ones the ATX conversion addressed.

1. Check the open redirect fix in `UserController.cs`

```
grep -A 5 "IsLocalUrl\|LocalRedirect" sourceCode/wildrydes.net/Controllers/UserController.cs
```

The original code had `return Redirect(Request.QueryString["or"])` which allowed open redirects. The converted code now uses `Url.IsLocalUrl()` validation and `LocalRedirect()` — **fixed** ✅

2. Check dependency injection is now in place

```
grep "public.*Controller(" sourceCode/wildrydes.net/Controllers/*.cs
```

All controllers now use **constructor-based dependency injection** with `DefaultContext`, `ILogger`, and `IConfiguration` — **fixed** ✅

3. Check structured logging is added

```
grep "_logger" sourceCode/wildrydes.net/Controllers/RideController.cs
```

The converted code uses ILogger<T> with structured logging (e.g., `_logger.LogError(ex, "DB Insert error: {Message}", ex.Message)`) — **fixed** ✅

4. Check EF Core migration with proper DbContext

```
cat sourceCode/wildrydes.net/Context/DefaultContext.cs
```

The DbContext now uses EF Core 8 with `DbContextOptions` constructor injection and `OnModelCreating` with owned entity types — **fixed** ✅

5. Review the summary of what was fixed vs what remains

Original Issue	Severity	Status
.NET Framework 4.8.1 legacy runtime	🔴 High	✅ Fixed — now .NET 8
ASP.NET MVC 5 superseded	🔴 High	✅ Fixed — now ASP.NET Core MVC
Entity Framework 6 superseded	🔴 High	✅ Fixed — now EF Core 8
Open redirect vulnerability	🔴 High	✅ Fixed — `Url.IsLocalUrl()` + `LocalRedirect()`
No dependency injection	🟡 Medium	✅ Fixed — constructor-based DI
No logging framework	🟡 Medium	✅ Fixed — ILogger<T> with structured logging
Modernizr 2.8.3 outdated	🟡 Medium	✅ Fixed — removed
WebGrease 1.6.0 unmaintained	🟡 Medium	✅ Fixed — removed

> **Note — Remaining items are intentional**
> The transformation is scoped as a **platform migration** preserving full functional equivalence. Some issues (e.g., SHA256 password hashing, missing CSRF on certain endpoints, session cookie hardening, no test coverage, business logic in controllers) are intentionally carried over. In a real-world scenario, you would address these by adjusting your transformation definition or as a follow-up phase using tools like Kiro after the migration is validated.

What You've Learned

In this section, you:

- Created a custom transformation definition tailored for .NET Framework 4.8.1 to .NET 8 migration
- Published the definition to the ATX registry for reuse across projects
- Executed the transformation with `atx custom def exec` using the named definition and `dotnet build` validation
- Verified the conversion: SDK-style `.csproj` targeting `net8.0`, `Program.cs` entry point, `appsettings.json`, EF Core 8, Tag Helpers, and a Linux-based Dockerfile
- All 20/20 exit criteria passed, covering build success, route preservation, database compatibility, AWS integration, and cross-platform operation
- Confirmed that 8 technical debt and security issues were automatically resolved, while remaining items are intentionally out of scope for a platform migration

In the next section, you'll deploy the modernized application. 🔄